```
while count < 3 and guess.lower() != name:    # .lower allows things like
 Guilherme to still match
    print("You are wrong!")
    guess = input("What is my name? ")
    count = count + 1

if guess.lower() != name:
    print("You are wrong!") # this message isn't printed in the third chance, so
 we print it now
    print("You ran out of chances.")
else:
    print("Yes! My name is", name + "!")
```

13 Dictionaries

This chapter is about dictionaries. Dictionaries have keys and values. The keys are used to find the values. Here is an example of a dictionary in use:

```
def print_menu():
    print('1. Print Phone Numbers')
    print('2. Add a Phone Number')
    print('3. Remove a Phone Number')
    print('4. Lookup a Phone Number')
    print('5. Quit')
    print()

numbers = {}
menu_choice = 0
print_menu()
while menu_choice != 5:
    menu_choice = int(input("Type in a number (1-5): "))
    if menu_choice == 1:
        print("Telephone Numbers:")
        for x in numbers.keys():
            print("Name: ", x, "\tNumber:", numbers[x])
        print()
    elif menu_choice == 2:
        print("Add Name and Number")
        name = input("Name: ")
        phone = input("Number: ")
        numbers[name] = phone
    elif menu_choice == 3:
        print("Remove Name and Number")
        name = input("Name: ")
        if name in numbers:
            del numbers[name]
        else:
            print(name, "was not found")
    elif menu_choice == 4:
        print("Lookup Number")
        name = input("Name: ")
        if name in numbers:
            print("The number is", numbers[name])
        else:
            print(name, "was not found")
    elif menu_choice != 5:
        print_menu()
```

And here is my output:

```
1. Print Phone Numbers
2. Add a Phone Number
3. Remove a Phone Number
4. Lookup a Phone Number
5. Quit

Type in a number (1-5): 2
Add Name and Number
```

```
Name: Joe
Number: 545-4464
Type in a number (1-5): 2
Add Name and Number
Name: Jill
Number: 979-4654
Type in a number (1-5): 2
Add Name and Number
Name: Fred
Number: 132-9874
Type in a number (1-5): 1
Telephone Numbers:
Name: Jill      Number: 979-4654
Name: Joe       Number: 545-4464
Name: Fred      Number: 132-9874

Type in a number (1-5): 4
Lookup Number
Name: Joe
The number is 545-4464
Type in a number (1-5): 3
Remove Name and Number
Name: Fred
Type in a number (1-5): 1
Telephone Numbers:
Name: Jill      Number: 979-4654
Name: Joe       Number: 545-4464

Type in a number (1-5): 5
```

This program is similar to the name list earlier in the chapter on lists. Here's how the program works. First the function `print_menu` is defined. `print_menu` just prints a menu that is later used twice in the program. Next comes the funny looking line `numbers = {}` . All that this line does is to tell Python that `numbers` is a dictionary. The next few lines just make the menu work. The lines

```
for x in numbers.keys():
    print("Name:", x, "\tNumber:", numbers[x])
```

go through the dictionary and print all the information. The function `numbers.keys()` returns a list that is then used by the `for` loop. The list returned by `keys()` is not in any particular order so if you want it in alphabetic order it must be sorted. Similar to lists the statement `numbers[x]` is used to access a specific member of the dictionary. Of course in this case `x` is a string. Next the line `numbers[name] = phone` adds a name and phone number to the dictionary. If `name` had already been in the dictionary `phone` would replace whatever was there before. Next the lines

```
if name in numbers:
    del numbers[name]
```

see if a name is in the dictionary and remove it if it is. The operator `name in numbers` returns true if `name` is in `numbers` but otherwise returns false. The line `del numbers[name]` removes the key `name` and the value associated with that key. The lines

```
if name in numbers:
    print("The number is", numbers[name])
```

check to see if the dictionary has a certain key and if it does prints out the number associated with it. Lastly if the menu choice is invalid it reprints the menu for your viewing pleasure.

A recap: Dictionaries have keys and values. Keys can be strings or numbers. Keys point to values. Values can be any type of variable (including lists or even dictionaries (those dictionaries or lists of course can contain dictionaries or lists themselves (scary right? :-)))). Here is an example of using a list in a dictionary:

```
max_points = [25, 25, 50, 25, 100]
assignments = ['hw ch 1', 'hw ch 2', 'quiz   ', 'hw ch 3', 'test']
students = {'#Max': max_points}

def print_menu():
    print("1. Add student")
    print("2. Remove student")
    print("3. Print grades")
    print("4. Record grade")
    print("5. Print Menu")
    print("6. Exit")

def print_all_grades():
    print('\t', end=' ')
    for i in range(len(assignments)):
        print(assignments[i], '\t', end=' ')
    print()
    keys = list(students.keys())
    keys.sort()
    for x in keys:
        print(x, '\t', end=' ')
        grades = students[x]
        print_grades(grades)

def print_grades(grades):
    for i in range(len(grades)):
        print(grades[i], '\t', end=' ')
    print()

print_menu()
menu_choice = 0
while menu_choice != 6:
    print()
    menu_choice = int(input("Menu Choice (1-6): "))
    if menu_choice == 1:
        name = input("Student to add: ")
        students[name] = [0] * len(max_points)
    elif menu_choice == 2:
        name = input("Student to remove: ")
        if name in students:
            del students[name]
        else:
            print("Student:", name, "not found")
    elif menu_choice == 3:
        print_all_grades()
    elif menu_choice == 4:
        print("Record Grade")
        name = input("Student: ")
        if name in students:
            grades = students[name]
            print("Type in the number of the grade to record")
            print("Type a 0 (zero) to exit")
            for i in range(len(assignments)):
                print(i + 1, assignments[i], '\t', end=' ')
            print()
            print_grades(grades)
            which = 1234
            while which != -1:
                which = int(input("Change which Grade: "))
                which -= 1    #same as which = which - 1
                if 0 <= which < len(grades):
```

```
                grade = int(input("Grade: "))
                grades[which] = grade
            elif which != -1:
                print("Invalid Grade Number")
        else:
            print("Student not found")
    elif menu_choice != 6:
        print_menu()
```

and here is a sample output:

```
1. Add student
2. Remove student
3. Print grades
4. Record grade
5. Print Menu
6. Exit

Menu Choice (1-6): 3
        hw ch 1        hw ch 2        quiz           hw ch 3        test
#Max    25             25             50             25             100

Menu Choice (1-6): 5
1. Add student
2. Remove student
3. Print grades
4. Record grade
5. Print Menu
6. Exit

Menu Choice (1-6): 1
Student to add: Bill

Menu Choice (1-6): 4
Record Grade
Student: Bill
Type in the number of the grade to record
Type a 0 (zero) to exit
1   hw ch 1     2   hw ch 2     3   quiz        4   hw ch 3     5   test
0               0               0               0               0
Change which Grade: 1
Grade: 25
Change which Grade: 2
Grade: 24
Change which Grade: 3
Grade: 45
Change which Grade: 4
Grade: 23
Change which Grade: 5
Grade: 95
Change which Grade: 0

Menu Choice (1-6): 3
       hw ch 1        hw ch 2        quiz           hw ch 3        test
#Max    25             25             50             25             100
Bill    25             24             45             23             95

Menu Choice (1-6): 6
```

Heres how the program works. Basically the variable `students` is a dictionary with the keys being the name of the students and the values being their grades. The first two lines just create two lists. The next line `students = {'#Max': max_points}` creates a new dictionary with the key {`#Max`} and the value is set to be `[25, 25, 50, 25, 100]`

(since thats what `max_points` was when the assignment is made) (I use the key `#Max` since `#` is sorted ahead of any alphabetic characters). Next `print_menu` is defined. Next the `print_all_grades` function is defined in the lines:

```
def print_all_grades():
    print('\t',end=" ")
    for i in range(len(assignments)):
        print(assignments[i], '\t',end=" ")
    print()
    keys = list(students.keys())
    keys.sort()
    for x in keys:
        print(x, '\t',end=' ')
        grades = students[x]
        print_grades(grades)
```

Notice how first the keys are gotten out of the `students` dictionary with the `keys` function in the line `keys = list(students.keys())`. `keys` is an iterable, and it is converted to list so all the functions for lists can be used on it. Next the keys are sorted in the line `keys.sort()`. `for` is used to go through all the keys. The grades are stored as a list inside the dictionary so the assignment `grades = students[x]` gives `grades` the list that is stored at the key `x`. The function `print_grades` just prints a list and is defined a few lines later.

The later lines of the program implement the various options of the menu. The line `students[name] = [0] * len(max_points)` adds a student to the key of their name. The notation `[0] * len(max_points)` just creates a list of 0's that is the same length as the `max_points` list.

The remove student entry just deletes a student similar to the telephone book example. The record grades choice is a little more complex. The grades are retrieved in the line `grades = students[name]` gets a reference to the grades of the student `name`. A grade is then recorded in the line `grades[which] = grade`. You may notice that `grades` is never put back into the students dictionary (as in no `students[name] = grades`). The reason for the missing statement is that `grades` is actually another name for `students[name]` and so changing `grades` changes `student[name]`.

Dictionaries provide an easy way to link keys to values. This can be used to easily keep track of data that is attached to various keys.

14 Using Modules

Here's this chapter's typing exercise (name it cal.py (`import` actually looks for a file named calendar.py and reads it in. If the file is named calendar.py and it sees a "import calendar" it tries to read in itself which works poorly at best.)):

```
import calendar
year = int(input("Type in the year number: "))
calendar.prcal(year)
```

And here is part of the output I got:

```
Type in the year number: 2001

                                  2001

       January                   February                  March

Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
1  2  3  4  5  6  7                1  2  3  4                1  2  3  4
8  9 10 11 12 13 14       5  6  7  8  9 10 11       5  6  7  8  9 10 11
15 16 17 18 19 20 21      12 13 14 15 16 17 18      12 13 14 15 16 17 18
22 23 24 25 26 27 28      19 20 21 22 23 24 25      19 20 21 22 23 24 25
29 30 31                  26 27 28                  26 27 28 29 30 31
```

(I skipped some of the output, but I think you get the idea.) So what does the program do? The first line `import calendar` uses a new command `import`. The command `import` loads a module (in this case the `calendar` module). To see the commands available in the standard modules either look in the library reference for python (if you downloaded it) or go to `http://docs.python.org/3/library/`. If you look at the documentation for the calendar module, it lists a function called `prcal` that prints a calendar for a year. The line `calendar.prcal(year)` uses this function. In summary to use a module `import` it and then use `module_name.function` for functions in the module. Another way to write the program is:

```
from calendar import prcal

year = int(input("Type in the year number: "))
prcal(year)
```

This version imports a specific function from a module. Here is another program that uses the Python Library (name it something like clock.py) (press Ctrl and the 'c' key at the same time to terminate the program):

```
from time import time, ctime

prev_time = ""
while True:
```

```
    the_time = ctime(time())
    if prev_time != the_time:
        print("The time is:", ctime(time()))
        prev_time = the_time
```

With some output being:

```
The time is: Sun Aug 20 13:40:04 2000
The time is: Sun Aug 20 13:40:05 2000
The time is: Sun Aug 20 13:40:06 2000
The time is: Sun Aug 20 13:40:07 2000

Traceback (innermost last):
 File "clock.py", line 5, in ?
    the_time = ctime(time())

KeyboardInterrupt
```

The output is infinite of course so I cancelled it (or the output at least continues until Ctrl+C is pressed). The program just does an infinite loop (`True` is always true, so `while True:` goes forever) and each time checks to see if the time has changed and prints it if it has. Notice how multiple names after the import statement are used in the line `from time import time, ctime`.

The Python Library contains many useful functions. These functions give your programs more abilities and many of them can simplify programming in Python.

14.0.1 Exercises

Rewrite the `high_low.py` program from section Decisions[1] to use an random integer between 0 and 99 instead of the hard-coded 7. Use the Python documentation to find an appropriate module and function to do this.

Solution

Rewrite `the high_low.py` program from section Decisions[2] to use an random integer between 0 and 99 instead of the hard-coded 7. Use the Python documentation to find an appropriate module and function to do this.

```
from random import randint
number = randint(0, 99)
guess = -1
while guess != number:
    guess = int(input ("Guess a number: "))
    if guess > number:
        print("Too high")
    elif guess < number:
            print("Too low")
```

1 Chapter 6.0.2 on page 32
2 Chapter 6.0.2 on page 32

```
print("Just right")
```

14.1 Other modules

Sometimes you want to use a python module that does not come with the Python installation. You can also import those, but you have to have them installed on your computer.

14.1.1 Creating your own module

When python reads the import command, it first checks files in your directory, then site-packages or pre installed modules. To make your own module, just create a .py file in the current directory and use the command:

```
import module
```

This will try to import the file module.py from your current directory and if not found, from site-packages and prepackaged modules. Changing module to the name of the .py file you created will import that file.

However, when it imports the module, it will basically start the file as a program, so any code on there will be run. You want to group all code into functions.

The `__name__` == `__main__` trick

In python, the variable `__name__` will give you the current name of the program. If a module you import prints the `__name__` variable, then it will print the name of the module. If the current file prints the `__name__` variable, it will print `__main__`, to show it is the main program.

If an if statement checks the name variable and runs code if the program is main, it can bypass the unintentional run problem created when a module is imported. Say for example you have a file, which runs some code. It also has a function you want to use in another program. However, you only want the function, not to run the code. By setting up the code below, it will only run the code if it is the file that was clicked on or started, not if it was imported.

```
if __name__ == '__main__':
    pass
```

In this instance, if the file is run but not imported, it will run the pass command. You can replace the pass command with the code you want to be run when not imported. Just remember to indent the code.

14.1.2 The pip module

The pip module is a module that comes with the python installation and acts as a module downloader/manager. You can download other modules from the internet with pip.

The pip module is not used in the python interpreter, but is run through the command line. To use it, open up your command line interpreter (for Windows it is Command Prompt, for Mac/Linux it is Terminal) and type in the following code:

```
py3 -m pip install module
```

or the alternate code

```
pip install module
```

This will try to download and install module from the user-submitted python modules database. Module can be changed to the name of the module.

15 More on Lists

We have already seen lists and how they can be used. Now that you have some more background I will go into more detail about lists. First we will look at more ways to get at the elements in a list and then we will talk about copying them.

Here are some examples of using indexing to access a single element of a list:

```
>>> some_numbers = ['zero', 'one', 'two', 'three', 'four', 'five']
»> some_numbers[0]
'zero'
»> some_numbers[4]
'four'
»> some_numbers[5]
'five'
```

All those examples should look familiar to you. If you want the first item in the list just look at index 0. The second item is index 1 and so on through the list. However what if you want the last item in the list? One way could be to use the `len()` function like `some_numbers[len(some_numbers) - 1]`. This way works since the `len()` function always returns the last index plus one. The second from the last would then be `some_numbers[len(some_numbers) - 2]`. There is an easier way to do this. In Python the last item is always index -1. The second to the last is index -2 and so on. Here are some more examples:

```
>>> some_numbers[len(some_numbers) - 1]
'five'
»> some_numbers[len(some_numbers) - 2]
'four'
»> some_numbers[-1]
'five'
»> some_numbers[-2]
'four'
»> some_numbers[-6]
'zero'
```

Thus any item in the list can be indexed in two ways: from the front and from the back.

Another useful way to get into parts of lists is using slicing. Here is another example to give you an idea what they can be used for:

```
>>> things = [0, 'Fred', 2, 'S.P.A.M.', 'Stocking', 42, "Jack", "Jill"]
»> things[0]
0
»> things[7]
'Jill'
»> things[0:8]
```

```
[0, 'Fred', 2, 'S.P.A.M.', 'Stocking', 42, 'Jack', 'Jill']
»> things[2:4]
[2, 'S.P.A.M.']
»> things[4:7]
['Stocking', 42, 'Jack']
»> things[1:5]
['Fred', 2, 'S.P.A.M.', 'Stocking']
```

Slicing is used to return part of a list. The slicing operator is in the form `things[first_index:last_index]`. Slicing cuts the list before the `first_index` and before the `last_index` and returns the parts in between. You can use both types of indexing:

```
>>> things[-4:-2]
['Stocking', 42]
»> things[-4]
'Stocking'
»> things[-4:6]
['Stocking', 42]
```

Another trick with slicing is the unspecified index. If the first index is not specified the beginning of the list is assumed. If the last index is not specified the whole rest of the list is assumed. Here are some examples:

```
>>> things[:2]
[0, 'Fred']
»> things[-2:]
['Jack', 'Jill']
»> things[:3]
[0, 'Fred', 2]
»> things[:-5]
[0, 'Fred', 2]
```

Here is a (HTML inspired) program example (copy and paste in the poem definition if you want):

```
poem = ["<B>", "Jack", "and", "Jill", "</B>", "went", "up", "the",
        "hill", "to", "<B>", "fetch", "a", "pail", "of", "</B>",
        "water.", "Jack", "fell", "<B>", "down", "and", "broke",
        "</B>", "his", "crown", "and", "<B>", "Jill", "came",
        "</B>", "tumbling", "after"]

def get_bold(text):
    true = 1
    false = 0
    ## is_bold tells whether or not we are currently looking at
    ## a bold section of text.
    is_bold = false
    ## start_block is the index of the start of either an unbolded
    ## segment of text or a bolded segment.
    start_block = 0
    for index in range(len(text)):
        ## Handle a starting of bold text
        if text[index] == "<B>":
            if is_bold:
                print("Error: Extra Bold")
            ## print "Not Bold:", text[start_block:index]
```